

Report

Assignment for the Seminar "Motion Control of Autonomous Robotic Vehicles"

Abstract

To be written

Modelling

The aim of this work is twofold: designing a controller for a rotary wing UAV vehicle and simulating its behaviour using the Python Library MuJoCo.

We start by stating the model used as reference. The UAV has four motors positioned in counterclock wise direction starting from the front right one. Each motor can be controlled individually by setting its forces f_1, f_2, f_3, f_4 . Individual rotor forces produce a wrench $w = [T, \tau_x, \tau_y, \tau_z]^\top$, which stacks together total thrust and torques w.r.t. to the three rotational degrees of freedom.

Additionally, we decide that the front left and back left rotors spin in counter wise direction and therefore have negative yaw reaction, while front left and back right rotors spin in the opposite direction and therefore impose positive yaw direction.

By Newton's Third Law, the motor exerts an equal and opposite reaction torque on the drone's body, thus Counter-Clockwise rotating motors exert a positive clockwise reaction torque on the frame while clockwise rotating motors exert a negative counter-clockwise reaction torque on the frame. Thus:

$$\tau_z = k_m(-f_1 + f_2 + f_3 - f_4)$$

Roll-torque, which acts with respect to the first rotational degree of freedom, is defined by the cross product of the lever arm and the force vector $\boldsymbol{\tau} = \mathbf{r} \times \mathbf{f}$. For an X-configuration quadcopter, motors 2 and 4 sit on the positive y -axis (left side), while motors 1 and 3 sit on the negative y -axis (right side), at a distance d from the center of mass. We can model this via the following equation:

$$\tau_x = d(-f_1 + f_2 - f_3 + f_4)$$

The reasoning for the pitch torque is quite similar: motors 3 and 4 sit on the negative x -axis, at the rear of the vehicle, and motors 1 and 2 sit at the front. Increasing rear motors (f_3, f_4) pitches the nose down/forward inducing a positive pitch torque while increasing front motors (f_1, f_2) pitches the nose up/backward inducing a negative pitch torque. We model this in the following way:

$$\tau_y = d(-f_1 - f_2 + f_3 + f_4)$$

From the kinematic point of view, the quadcopter is assumed to be a point mass, therefore $\dot{\mathbf{p}} = \mathbf{v}$, the attitude is modelled via a rotation matrix R and the body angular velocities are $\Omega = [\Omega_x, \Omega_y, \Omega_z]^\top$, they are governed by: $\dot{R} = R\hat{\Omega}$ where $\hat{\Omega}$ is the skew-symmetric cross-product matrix.

We assume that the governing differential equation is:

$$m \ddot{\mathbf{p}} = m \mathbf{g} \mathbf{e}_3 - TR \mathbf{e}_3$$

where m is the mass of the quadcopter, $\mathbf{g}$ is the gravity vector, $\mathbf{e}$ is the unit vector, T is the thrust and R is the orientation matrix converting the body frame to the world frame, representing the orientation of the quadcopter.

We model the rotational dynamics using the classical Newton-Euler equations for a rigid body rotating in space:

$$\dot{R} = R \hat{\Omega} \quad J\dot{\Omega} + \Omega \times J\Omega = \boldsymbol{\tau}$$

where J is the 3×3 rigid-body inertia tensor, Ω is the angular velocity vector in the body-fixed frame, $\boldsymbol{\tau}$ is the net control torque vector applied by the rotors.

Task

The task is for the quadcopter to elevate track an horizontal circle:

$$\mathbf{p}_d(t) = [R \cos(\omega t), R \sin(\omega t), z_d]^\top,$$

of radius of 3.0 m with angular velocity of $\omega = 0.5$ rad/s and of elevation $z_d = 3$ m.

The aim of the work is to design a controller that is certified to be Globally Asymptotically Stable.

Assumptions

For the controller to be certifiable, the motors cannot have saturation and therefore may produce arbitrary forces, also negative once. Secondly, this work focuses on simulating a gentle trajectory: we decide to track a circular trajectory with constant angular rate ω , which implies that

$$\dot{\Omega}_d = 0$$

This assumption is required to model a certifiable controller that takes dynamics into account.

Controller

The designed controller is a two-tier geometric cascade. Quadrotors are underactuated—they have 6 degrees of freedom but only 4 actuators. They cannot translate without first rotating.

To solve this, the controller relies on time-scale separation:

- outer loop (Low-bandwidth control): computes desired forces and desired orientations based on position errors;
- inner loop (High-bandwidth control): computes motor torques to track the desired orientation.

The proof of convergence assumes that the inner loop converges so quickly that the outer loop can treat orientation changes as essentially instantaneous.

On the kinematic, we'll use a simple derivative control on the velocity error. We start by considering the position error $\mathbf{e} = \mathbf{p} - \mathbf{p}^d$: its derivation equals to $\dot{\mathbf{e}} = \mathbf{v} - \mathbf{v}^d$. We want that the error decreases over time. The outer loop controls velocity and therefore assumes to have complete control over it, therefore $\dot{\mathbf{x}} - \dot{\mathbf{x}}^d = \mathbf{u} - \mathbf{v}^d$. We choose a control such that: $\mathbf{u} - \mathbf{v}^d = -K\mathbf{e}^p$.

This is a linear, decoupled, first-order system, therefore each component satisfies $\dot{e}_i = -k_i e_i$, with the known solution $e_i(t) = e_i(0)e^{-k_i t} \rightarrow 0$.

Our control $\mathbf{u}$ will be the $v_{\text{cmd}} = \mathbf{v}^d - K\mathbf{e}^p$

The attitude of the vehicle is represented by a rotation matrix $R \in SO(3)$, the Lie group of 3×3 orthogonal matrices with determinant +1. The desired attitude is $R_d \in SO(3)$, computed from the desired thrust vector and yaw.

The attitude tracking error is defined on $SO(3)$ as (Lee et al. 2010, eq. 7):

$$\mathbf{e}_R = \frac{1}{2} \text{vee}(R_d^\top R - R^\top R_d) \in \mathbf{R}^3$$

where $\text{vee}(\cdot)$ extracts the axial vector from a skew-symmetric matrix. This error is zero if and only if $R = R_d$.

This form is particularly useful when proving GAS in Lyapunov analysis.

Lyapunov Proof

0.1 Symbolic computation

Here, write why are the symbolic simulations being carried out.

0.2 Tier 1 - Kinematic error

The Theory: The system needs to track a target trajectory $p_d(t)$. You define a simple tracking error $\mathbf{e} = [e_x, e_y, e_z, e_\psi]^\top$. To prove this error shrinks to zero, you define a Lyapunov candidate function that represents the "error energy" of the position loop:

$$V_1 = \frac{1}{2}e_x^2 + \frac{1}{2}e_y^2 + \frac{1}{2}e_z^2 + \frac{1}{2}e_\psi^2$$

For this to be stable, the energy must constantly dissipate. By commanding a velocity $\mathbf{v}_{\text{cmd}} = \mathbf{v}_d - K\mathbf{e}$, the system forces the error dynamics to become $\dot{\mathbf{e}} = -K\mathbf{e}$.

When the notebook calculates the time derivative of the Lyapunov function ($\dot{V}_1 = \nabla V_1 \cdot \dot{\mathbf{e}}$), it results in:

$$\dot{V}_1 = -k_x e_x^2 - k_y e_y^2 - k_z e_z^2 - k_\psi e_\psi^2$$

The Conclusion: Because your gain matrix K is positive definite ($k_i > 0$), $\dot{V}_1$ is strictly **Negative Definite (ND)**. The error energy strictly decays to exactly zero. Therefore, the outer loop is **Globally Asymptotically Stable (GAS)**.

0.3 Tier 2 - Attitude inner loop

Following Lee et al. (2010, eq. 9), define:

$$V_2(\mathbf{e}_R, \mathbf{e}_\Omega) = k_R \Psi(R, R_d) + \frac{1}{2} \mathbf{e}_\Omega^\top J \mathbf{e}_\Omega$$

where $\Psi(R, R_d) = \frac{1}{2} \text{tr}(I - R_d^\top R)$ is the **attitude tracking function** on $SO(3)$, a metric-like quantity that is zero when $R = R_d$ and positive otherwise.

Near the equilibrium, $\Psi \approx \frac{1}{4} \|\mathbf{e}_R\|^2$ (Lee et al. Lemma 1), giving the quadratic approximation:

$$V_2 \approx \frac{k_R}{4} \|\mathbf{e}_R\|^2 + \frac{1}{2} \mathbf{e}_\Omega^\top J \mathbf{e}_\Omega$$

Positive Definiteness: Both terms are non-negative; they vanish simultaneously only when $\mathbf{e}_R = \mathbf{0}$ and $\mathbf{e}_\Omega = \mathbf{0}$. Since $k_R > 0$ and $J \succ 0$ (physically), V_2 is positive definite. Add the gyroscopic term to the torque law (Lee et al. 2010, eq. 16):

$$\boldsymbol{\tau}_{\text{cert}} = -k_R \mathbf{e}_R - k_w \mathbf{e}_\Omega + \boldsymbol{\Omega} \times J \boldsymbol{\Omega}$$

The angular dynamics become:

$$J \dot{\boldsymbol{\Omega}} = -k_R \mathbf{e}_R - k_w \mathbf{e}_\Omega + \boldsymbol{\Omega} \times J \boldsymbol{\Omega} - \boldsymbol{\Omega} \times J \boldsymbol{\Omega} = -k_R \mathbf{e}_R - k_w \mathbf{e}_\Omega$$

The gyroscopic term in the torque **exactly cancels** the gyroscopic term in Euler's equation. The angular error dynamics simplify to:

$$J \dot{\mathbf{e}}_\Omega \approx -k_R \mathbf{e}_R - k_w \mathbf{e}_\Omega$$

Now the time derivative of V_2 is (Lee et al. 2010, eq. 14):

$$\dot{V}_2|_{\text{cert}} = k_R \mathbf{e}_R^\top \dot{\mathbf{e}}_R + \mathbf{e}_\Omega^\top J \dot{\mathbf{e}}_\Omega$$

Using the $SO(3)$ kinematics $\dot{\mathbf{e}}_R \approx \mathbf{e}_\Omega$ near equilibrium and the simplified angular dynamics:

$$\begin{aligned} \dot{V}_2|_{\text{cert}} &= k_R \mathbf{e}_R^\top \mathbf{e}_\Omega + \mathbf{e}_\Omega^\top (-k_R \mathbf{e}_R - k_w \mathbf{e}_\Omega) \\ &= k_R \mathbf{e}_R^\top \mathbf{e}_\Omega - k_R \mathbf{e}_\Omega^\top \mathbf{e}_R - k_w \|\mathbf{e}_\Omega\|^2 \\ &= -k_w \|\mathbf{e}_\Omega\|^2 \leq 0 \end{aligned}$$

Negative Semi-Definiteness: $\dot{V}_2|_{\text{cert}} \leq 0$. (zero only when $\mathbf{e}_\Omega = \mathbf{0}$)

3.5 LaSalle for the Attitude Loop

Since $\dot{V}_2$ is only **negative semi-definite** (not ND — it is zero whenever $\mathbf{e}_\Omega = \mathbf{0}$ regardless of $\mathbf{e}_R$), we must invoke LaSalle.

The set where $\dot{V}_2 = 0$:

$$\mathcal{S}_{\text{att}} = \{(\mathbf{e}_R, \mathbf{e}_\Omega) : k_w \|\mathbf{e}_\Omega\|^2 = 0\} = \{(\mathbf{e}_R, \mathbf{0})\}$$

Now we find the **largest invariant set** $\mathcal{M}_{\text{att}} \subseteq \mathcal{S}_{\text{att}}$. On $\mathcal{S}_{\text{att}}$ we have $\mathbf{e}_\Omega = \mathbf{0}$ and therefore $\dot{\mathbf{e}}_\Omega = \mathbf{0}$. From the angular error dynamics:

$$J \dot{\mathbf{e}}_\Omega = -k_R \mathbf{e}_R - k_w \mathbf{e}_\Omega = -k_R \mathbf{e}_R = \mathbf{0}$$

Since $k_R > 0$, this requires $\mathbf{e}_R = \mathbf{0}$. Therefore:

$$\mathcal{M}_{\text{att}} = \{(\mathbf{0}, \mathbf{0})\}$$

By LaSalle's Invariance Principle, all bounded trajectories converge to $\mathcal{M}_{\text{att}}$.

Architecture of the coding base

TODO talk about mcav_project, on how is structured. Take as a start the following structure

and the following constraints :

In this project, we will work with the quadricopter.

In this project we will use PyBullet for the simulations and as a physics engine.

There's a need to create a good environment to work with. We want to have a modular environment.

There will be notebooks and there will be files with function.

There will also be the possibility of working inside a wrapper (which are classes): the first wrapper sets up the PyBullet environment WITH the creation of a video, the second wrapper will not have videos.

Each wrapper should have an overridable function, which represents the simulation loop. In this loop, it will be possible to write data in a data structure, maybe an array of json objects: in that case, there should be a function which allows me to convert that json structure into a file (to be saved) and into a table.

At each simulation step, the json data structure should be saved.

Also, each process that requires time should have a tqdm so that progress can be tracked.

Video FPS and Simulation FPS should ALWAYS be distinct and separable.

The idea is that we want to do Lyapunov control with one or more vehicles in formation. The most complex task that we could achieve is cooperative vehicle target tracking, so our setup should support that.

ROS is not required.

Also, the whole setup should work very well on ARM mac (M3)

So, we want to do symbolic computations, we want to verify convergence, do a lot a plot, do simulations with PyBullet and finally create a latex report.

This will be an iterative work, we will start with the unicycle and doing path-following for the unicycle. Then we will move iteratively to more complex things.

Finally, I want to be able to change parameters quite easily and also do optimization.

All of this section should not be longer than section (0.3 Tier 2 - Attitude inner loop)

Controller implementation

TODO; reference code snipped with the controllers (both controllers) and reference it here.

Simulation

TODO 1 : reference code where you set up simulations (create classes, etc...) and then put the code at the end of this notebook

VERY BRIEF Section on path following - trajectory tracking

Results

Conclusion
